Apache Airflow Architecture & Guide

From manual morning executions to robust, automated Python orchestration

> **The Problem:** Running pipelines manually every morning wastes time and invites human error. Airflow automates execution, manages task dependencies, and builds resilient pipelines.

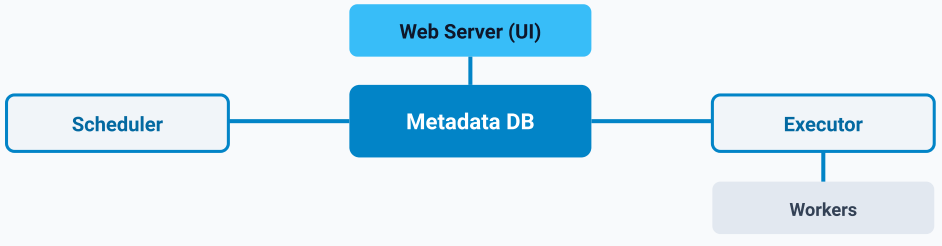
1. WHAT IS APACHE AIRFLOW?

Created by Airbnb, Apache Airflow is an open-source workflow orchestration platform written in Python. It allows data engineers to programmatically author, schedule, and monitor complex data workflows (ETL/ELT, ML pipelines, batch analytics) using standard Python code.

2. CORE ARCHITECTURE (6 KEY COMPONENTS)

COMPONENT	ROLE & CORE RESPONSIBILITY
Web Server (UI)	Provides a visual dashboard to monitor DAGs, inspect logs, trigger manual runs, and review state histories.
Scheduler	Parses Python DAG files, monitors scheduled intervals, evaluates dependencies, and submits tasks to the Executor.
Executor	Defines *how* tasks are run (Sequential, Local, Celery, or Kubernetes) and distributes work allocations.
Workers	The execution units that pull individual task instances from the queue and execute the Python code or commands.
Metadata DB	Central database (PostgreSQL/MySQL) storing all task states, schedules, variables, connections, and metadata.
DAG Folder	Directory containing Python workflow scripts, continuously scanned by the Scheduler and Web Server.

3. ARCHITECTURE DATA FLOW



4. DIRECTED ACYCLIC GRAPHS (DAGS)

Workflows in Airflow are represented as **DAGs**. A DAG defines tasks and their dependencies programmatically. They are *Directed* (clear downstream execution path) and *Acyclic* (no circular loops allowed).

```
from datetime import datetime
from airflow import DAG
from airflow.operators.python import PythonOperator

with DAG(
    dag_id="etl_pipeline",
    start_date=datetime(2024, 1, 1),
    schedule_interval="@daily",
    catchup=False
) as dag:

    t1 = PythonOperator(task_id="extract", python_callable=extract)
    t2 = PythonOperator(task_id="transform", python_callable=transform)
    t3 = PythonOperator(task_id="load", python_callable=load)

    # Dependency rule: extract -> transform -> load
    t1 >> t2 >> t3
```

5. EXECUTOR TYPES & COMMON OPERATORS

Executors (Execution Environment)

SequentialExecutor: Dev testing; 1 task at a time.
LocalExecutor: Parallel tasks on 1 machine.
CeleryExecutor: Distributed worker cluster.
KubernetesExecutor: Pod per task auto-scaling.

Common Airflow Operators

PythonOperator: Executes Python functions.
BashOperator: Runs bash commands/scripts.
SparkSubmitOperator: Submits Spark jobs.
BranchPythonOperator: Conditional branching.

6. TASK STATE LIFECYCLE & INTERVIEW QUESTION

none → scheduled → queued → running → [success | failed (up_for_retry) | skipped]

Scenario: *"How do you ensure Extract → Transform → Load runs sequentially and fails safely if Extract drops?"*

Answer: Define tasks with `PythonOperator` and set dependencies using `t1 >> t2 >> t3`. Airflow defaults to `all_success` rules, automatically skipping downstream tasks (`t2`, `t3`) if `t1` fails. Add `retries=3` in default args for failure recovery.

> ***Key Takeaway:*** *Junior engineers trigger pipelines manually. Senior engineers orchestrate them with Airflow code.*